

CO2_AT3

Debugging Brute Force String Matching Algorithm

Name: P C Vamsidhar Reddy

Reg no: 192465045

Aim

To debug the brute-force string matching algorithm by correcting the loop bounds, ensuring every valid alignment is checked, and analyzing its time complexity.

Problem in the Given Code

The original code uses:

```
for i in range(n - m):
```

Correction

Replace

```
for i in range(n - m):
```

with

```
for i in range(n - m + 1):
```

Procedure

1. Read the input text and pattern.
2. Find the lengths of the text and pattern.
3. Compare the pattern with every valid position in the text.
4. If all characters match, store the starting index.
5. Display all matching positions.
6. Analyze the time complexity.

Algorithm

Step 1: Start.

Step 2: Read the text and pattern.

Step 3: Calculate the lengths of the text and pattern.

Step 4: For each valid starting position from 0 to $n - m$, compare the pattern with the text.

Step 5: If all characters match, store the index.

Step 6: Continue until all positions are checked.

Step 7: Display the matching positions.

Step 8: Stop.

Analysis

- Every valid alignment of the pattern is checked.
- The corrected loop ensures the final possible position is included.
- In the worst case, each alignment compares all characters of the pattern.

Worst-case Time Complexity: $\Theta(n \times m)$

Space Complexity: $\Theta(k)$, where k is the number of matches stored.

Result

The loop boundary was corrected from **range($n - m$)** to **range($n - m + 1$)**, ensuring that every valid starting position is examined. The corrected brute-force string matching algorithm successfully finds all occurrences of the pattern in the text while maintaining a time complexity of $\Theta(n \times m)$.